COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S4
DATE PERFORMED:	8/28/26	**DATE SUBMITTED:**	8/28/26
NAME:	Reyes, Cyrill Joferson B.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. ___ [ACTIVITY NAME]

1. PROCEDURE OUTPUT

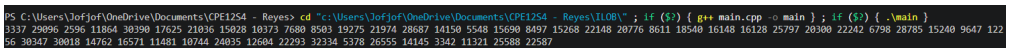
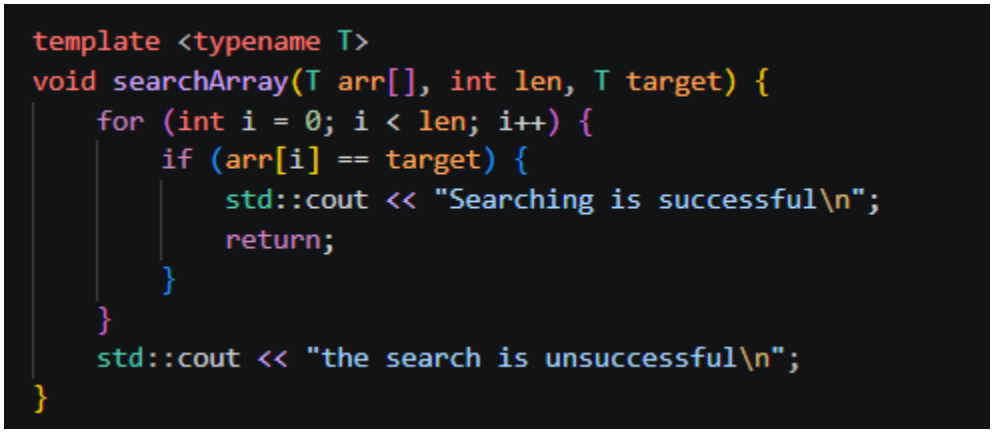
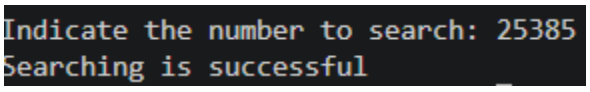
Screenshot	
Observations	**The rand() function is used to generate pseudo-random integers and set the seed with srand(time(0)), but since the numbers are in a random array of size 50, binary search cannot be performed immediately.**

Table 6-1. Data Generated and Observations.

Code	
Output	
Observations	**It runs through each element's index, one at 0 and one on the negative side (i.e. It can handle both sorted and unsorted arrays, and in the worst-case scenario, it necessitates O(n) time.**

```
template <typename T>
void searchArray(T arr[], int len, T target) {
    for (int i = 0; i < len; i++) {
        if (arr[i] == target) {
            std::cout << "Searching is successful\n";
            return;
        }
    }
    std::cout << "the search is unsuccessful\n";
}
```


Table 6-2a. Linear Search for Arrays

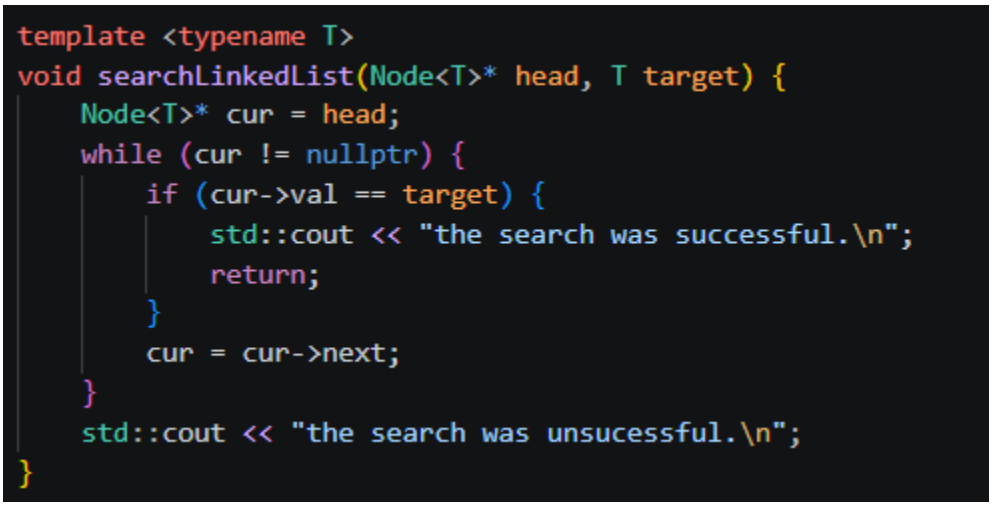
Code	
Output	
Observations	**Continuates with the node pointers, such as cur = cur->next, while progressing through each linked nodes in sequence. By examining different memory spaces that contain "KITTEN" characters, the device operates in a time-varying manner that increases with the size of the data.**

Table 6-2b. Linear Search for Linked List

Code

```cpp
const int MAX_ITEMS = 50;

int main() {
    // 1. Array Linear Search Test
    int numbers[MAX_ITEMS];
    srand(time(0));
    for (int idx = 0; idx < MAX_ITEMS; idx++) {
        numbers[idx] = rand();
        std::cout << numbers[idx] << " ";
    }
    std::cout << std::endl;

    int userNum;
    std::cout << "Indicate the number to search: ";
    std::cin >> userNum;
    searchArray(numbers, MAX_ITEMS, userNum);

    // 2. Linear Search Linked List (Unsorted word: K-I-T-T-E-N)
    Node<char>* kittenList = new Node<char>('K');
    kittenList->next = new Node<char>('I');
    kittenList->next->next = new Node<char>('T');
    kittenList->next->next->next = new Node<char>('T');
    kittenList->next->next->next->next = new Node<char>('E');
    kittenList->next->next->next->next->next = new Node<char>('N');

    char userChar;
    std::cout << "Indicate the letter to search: ";
    std::cin >> userChar;
    searchLinkedList(kittenList, userChar);
```

Output

```
Indicate the letter to search: K
the search was successful.
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49
searching is success!
```

Observations

Table 6-3a. Binary Search for Arrays

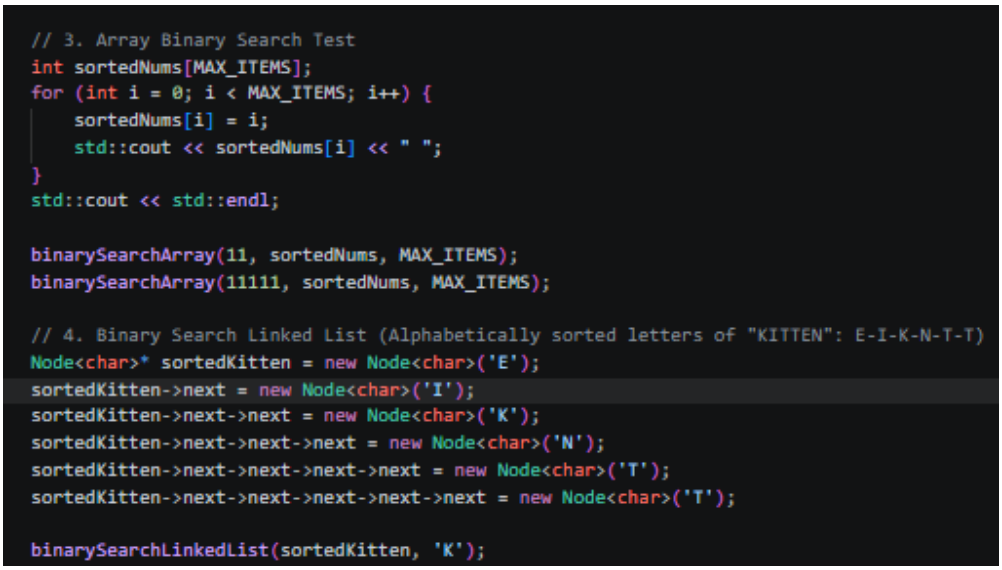
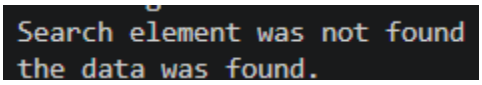
Code	
Output	
Observations	

Table 6-3b. Binary Search for Linked List

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Problem 1. Suppose you are doing a sequential search of the list [15, 18, 2, 19, 18, 0, 8, 14, 19, 14]. Utilizing both a linked list and an array approach to the list, use sequential search and identify how many comparisons
would be necessary to find the key '18'?

```cpp
// --- PROBLEM 1: Sequential Search with Comparison Tracking ---
template <typename T>
void problem1_array(T arr[], int len, T target) {
    int comparisons = 0;
    for (int i = 0; i < len; i++) {
        comparisons++;
        if (arr[i] == target) {
            std::cout << "[Array] Key " << target << " found at index " << i
                      << " in " << comparisons << " comparison(s).\n";
            return;
        }
    }
    std::cout << "[Array] Key not found after " << comparisons << " comparison(s).\n";
}

template <typename T>
void problem1_ll(Node<T>* head, T target) {
    int comparisons = 0;
    Node<T>* cur = head;
    while (cur != nullptr) {
        comparisons++;
        if (cur->val == target) {
            std::cout << "[Linked List] Key " << target << " found in "
                      << comparisons << " comparison(s).\n";
            return;
        }
        cur = cur->next;
    }
    std::cout << "[Linked List] Key not found after " << comparisons << " comparison(s).\n";
}
```

```
PS C:\Users\Jofjof\OneDrive\Documents\CPE12S4 - Reyes> cd "c
=== PROBLEM 1: Sequential Search Comparisons (Key: 18) ===
[Array] Key 18 found at index 1 in 2 comparison(s).
[Linked List] Key 18 found in 2 comparison(s).
```

Problem 2. Modify your sequential search algorithm so that it returns the count of repeating instances for a given search element 'k'. Test on the same list given in problem 1.

```cpp
// --- PROBLEM 2: Occurrence Counting ---
template <typename T>
int problem2_array(T arr[], int len, T target) {
    int count = 0;
    for (int i = 0; i < len; i++) {
        if (arr[i] == target) count++;
    }
    return count;
}

template <typename T>
int problem2_ll(Node<T>* head, T target) {
    int count = 0;
    Node<T>* cur = head;
    while (cur != nullptr) {
        if (cur->val == target) count++;
        cur = cur->next;
    }
    return count;
}
```

```
=== PROBLEM 2: Occurrence Counting (Key: 18) ===
[Array] Occurrences of 18: 2
[Linked List] Occurrences of 18: 2
```

Problem 3. Suppose you have the following sorted list [3, 5, 6, 8, 11, 12, 14, 15, 17, 18] and are using the binary search algorithm. If you wanted to find the key 8, draw a diagram that shows how the searching works
per iteration of the algorithm. Prove that your drawing is correct by implementing the algorithm and showing a
screenshot of the code and the output console.

```cpp
// --- PROBLEM 3: Iterative Binary Search with Trace Output ---
template <typename T>
void problem3_binarySearchIterative(T target, T arr[], int len) {
    int left = 0, right = len - 1;
    int iteration = 1;
    while (left <= right) {
        int mid = left + (right - left) / 2;
        std::cout << " Iteration " << iteration++ << ": left=" << left << ", right=" << right
                  << ", mid=" << mid << " (val=" << arr[mid] << ")\n";

        if (arr[mid] == target) {
            std::cout << " Success! Key " << target << " found at index " << mid << ".\n";
            return;
        }
        if (arr[mid] < target) left = mid + 1;
        else right = mid - 1;
    }
    std::cout << " Key " << target << " not found.\n";
}
```

```
=== PROBLEM 3: Iterative Binary Search Trace (Key: 8) ===
 Iteration 1: left=0, right=9, mid=4 (val=11)
 Iteration 2: left=0, right=3, mid=1 (val=5)
 Iteration 3: left=2, right=3, mid=2 (val=6)
 Iteration 4: left=3, right=3, mid=3 (val=8)
 Success! Key 8 found at index 3.
```

Problem 4. Modify the binary search algorithm so that the algorithm becomes recursive. Using this new recursive binary search, implement a solution to the same problem for problem 3

```cpp
// --- PROBLEM 4: Recursive Binary Search with Call Trace ---
template <typename T>
int problem4_binarySearchRecursive(T target, T arr[], int left, int right, int callDepth = 1) {
    if (left > right) {
        std::cout << " Key " << target << " not found.\n";
        return -1;
    }

    int mid = left + (right - left) / 2;
    std::cout << " Call " << callDepth << ": bounds [" << left << ", " << right
              << "], mid=" << mid << " (val=" << arr[mid] << ")\n";

    if (arr[mid] == target) {
        std::cout << " Success! Key " << target << " found at index " << mid << ".\n";
        return mid;
    }

    if (arr[mid] < target) {
        return problem4_binarySearchRecursive(target, arr, mid + 1, right, callDepth + 1);
    } else {
        return problem4_binarySearchRecursive(target, arr, left, mid - 1, callDepth + 1);
    }
}
```

```
=== PROBLEM 4: Recursive Binary Search Trace (Key: 8) ===
 Call 1: bounds [0, 9], mid=4 (val=11)
 Call 2: bounds [0, 3], mid=1 (val=5)
 Call 3: bounds [2, 3], mid=2 (val=6)
 Call 4: bounds [3, 3], mid=3 (val=8)
 Success! Key 8 found at index 3.
```

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.